

Experiment 3

Aim: To create an Employee–Department–Project database schema, insert at least 30 employees across 5 departments and 8 projects, and execute SQL queries demonstrating selection, projection, aggregation, GROUP BY, HAVING, CASE expressions, and ORDER BY.

Source Code:

```
-- =====
-- 1. CREATE DEPARTMENT TABLE
-- =====

CREATE TABLE Department (
    dept_id INT PRIMARY KEY,
    dept_name VARCHAR(50),
    location VARCHAR(50)
);

-- =====
-- 2. CREATE EMPLOYEE TABLE
-- =====

CREATE TABLE Employee (
    emp_id INT PRIMARY KEY,
    emp_name VARCHAR(50),
    salary DECIMAL(10,2),
    age INT,
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
);

-- =====
-- 3. CREATE PROJECT TABLE
-- =====

CREATE TABLE Project (
    project_id INT PRIMARY KEY,
    project_name VARCHAR(100),
    budget DECIMAL(12,2),
    dept_id INT,
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)
```

);

-- =====

-- 4. INSERT 5 DEPARTMENTS

-- =====

INSERT INTO Department VALUES

(1, 'CSE', 'Delhi'),

(2, 'AIML', 'Noida'),

(3, 'HR', 'Lucknow'),

(4, 'Finance', 'Mumbai'),

(5, 'Marketing', 'Bangalore');

-- =====

-- 5. INSERT 30 EMPLOYEES

-- =====

INSERT INTO Employee VALUES

(101, 'Rahul Sharma', 55000, 24, 1),

(102, 'Aman Verma', 62000, 26, 1),

(103, 'Neha Gupta', 58000, 25, 1),

(104, 'Rohit Singh', 72000, 29, 1),

(105, 'Priya Mishra', 68000, 27, 1),

(106, 'Arjun Yadav', 49000, 23, 1),

(107, 'Ananya Kapoor', 75000, 28, 2),

(108, 'Vivek Patel', 82000, 30, 2),

(109, 'Sneha Jain', 65000, 26, 2),

(110, 'Karan Mehta', 91000, 32, 2),

(111, 'Simran Kaur', 70000, 27, 2),

(112, 'Aditya Rao', 60000, 25, 2),

(113, 'Pooja Singh', 45000, 24, 3),

(114, 'Nitin Kumar', 52000, 28, 3),

(115, 'Riya Sharma', 58000, 29, 3),

(116, 'Mohit Verma', 63000, 31, 3),

(117, 'Kavita Gupta', 55000, 26, 3),

(118, 'Sahil Khan', 48000, 25, 3),

(119, 'Deepak Agarwal', 78000, 30, 4),

(120, 'Megha Joshi', 85000, 29, 4),

```
(121, 'Varun Malhotra', 95000, 34, 4),  
(122, 'Isha Sinha', 67000, 27, 4),  
(123, 'Manish Tiwari', 73000, 31, 4),  
(124, 'Shreya Roy', 88000, 32, 4),  
(125, 'Akash Gupta', 52000, 24, 5),  
(126, 'Nisha Patel', 61000, 26, 5),  
(127, 'Yash Sharma', 69000, 28, 5),  
(128, 'Tanya Verma', 74000, 29, 5),  
(129, 'Harsh Singh', 57000, 25, 5),  
(130, 'Aditi Mehra', 80000, 30, 5);
```

```
-- =====
```

```
-- 6. INSERT 8 PROJECTS
```

```
-- =====
```

```
INSERT INTO Project VALUES
```

```
(201, 'E-Commerce Platform', 500000, 1),  
(202, 'Cyber Security System', 750000, 1),  
(203, 'AI Chatbot', 900000, 2),  
(204, 'Machine Learning Model', 850000, 2),  
(205, 'Employee Management System', 400000, 3),  
(206, 'Financial Analysis Tool', 650000, 4),  
(207, 'Marketing Automation', 550000, 5),  
(208, 'Customer Analytics', 700000, 5);
```

OUTPUT:

Selection: Display employees with salary greater than 70000.

```
SELECT *  
  
FROM Employee  
  
WHERE salary > 70000;
```

The screenshot shows the ByteXL IDE interface. On the left, a SQL editor contains a query to select employee names and salaries from the 'Employee' table where the salary is greater than 70000. The query is as follows:

```

-- 6. INSERT 8 PROJECTS
-- =====
76 INSERT INTO Project VALUES
77 (201, 'E-Commerce Platform', 500000, 1),
78 (202, 'Cyber Security System', 750000, 1),
79 (203, 'AI Chatbot', 900000, 2),
80 (204, 'Machine Learning Model', 850000, 2),
81 (205, 'Employee Management System', 400000, 3),
82 (206, 'Financial Analysis Tool', 650000, 4),
83 (207, 'Marketing Automation', 550000, 5),
84 (208, 'Customer Analytics', 700000, 5);
85 SELECT *
86 FROM Employee
87 WHERE salary > 70000;

```

On the right, the output of the query is displayed in a table format:

emp_id	emp_name	salary	age	dept_id
104	Rohit Singh	72000.00	29	1
107	Ananya Kapoor	75000.00	28	2
108	Vivek Patel	82000.00	30	2
110	Karan Mehta	91000.00	32	2
119	Deepak Agarwal	78000.00	30	4
120	Megha Joshi	65000.00	29	4
121	Varun Malhotra	95000.00	34	4
123	Manish Tiwari	73000.00	31	4
124	Shreya Roy	80000.00	32	4
126	Nisha Patel	61000.00	26	5
127	Yash Sharma	69000.00	28	5
128	Tanya Verma	74000.00	29	5
129	Harsh Singh	57000.00	25	5
130	Aditi Mehra	80000.00	30	5

Projection: Display employee names and salaries.

SELECT emp_name, salary

FROM Employee;

The screenshot shows the ByteXL IDE interface. On the left, a SQL editor contains a query to select employee names and salaries from the 'Employee' table. The query is as follows:

```

-- 6. INSERT 8 PROJECTS
-- =====
76 INSERT INTO Project VALUES
77 (201, 'E-Commerce Platform', 500000, 1),
78 (202, 'Cyber Security System', 750000, 1),
79 (203, 'AI Chatbot', 900000, 2),
80 (204, 'Machine Learning Model', 850000, 2),
81 (205, 'Employee Management System', 400000, 3),
82 (206, 'Financial Analysis Tool', 650000, 4),
83 (207, 'Marketing Automation', 550000, 5),
84 (208, 'Customer Analytics', 700000, 5);
85 SELECT emp_name, salary
86 FROM Employee;

```

On the right, the output of the query is displayed in a table format:

emp_name	salary
Rahul Sharma	55000.00
Aman Verma	62000.00
Neha Gupta	58000.00
Rohit Singh	72000.00
Priya Mishra	68000.00
Arjun Yadav	49000.00
Ananya Kapoor	75000.00
Vivek Patel	82000.00
Karan Mehta	91000.00
Simran Kaur	70000.00
Aditya Rao	60000.00
Pooja Singh	45000.00
Nitin Kumar	52000.00
Riya Sharma	58000.00
Mohit Verma	63000.00
Kavita Gupta	55000.00
Sahil Khan	48000.00
Deepak Agarwal	78000.00
Megha Joshi	65000.00
Varun Malhotra	95000.00
Isha Sinha	67000.00
Manish Tiwari	73000.00
Shreya Roy	80000.00
Akash Gupta	52000.00
Nisha Patel	61000.00
Yash Sharma	69000.00

Aggregation: Count employees and calculate average salary

SELECT COUNT(*) AS total_employees,

AVG(salary) AS average_salary FROM Employee;

The screenshot shows the byteXL IDE interface. The left pane displays a MySQL query: `SELECT dept_id, COUNT(*) AS employee_count FROM Employee GROUP BY dept_id HAVING COUNT(*) > 5;`. The right pane shows the output of the query, which is a table with two columns: `dept_id` and `employee_count`. The output table has five rows, each representing a department with more than five employees.

dept_id	employee_count
1	6
2	6
3	6
4	6
5	6

Having: departments having more than 5 employees

SELECT dept_id,

COUNT(*) AS employee_count FROM Employee

GROUP BY dept_id HAVING COUNT(*) > 5;

The screenshot shows the byteXL IDE interface. The left pane displays a MySQL query: `SELECT emp_name, salary, CASE WHEN salary >= 75000 THEN 'High Salary' WHEN salary >= 60000 THEN 'Medium Salary' ELSE 'Low Salary' END AS salary_category FROM Employee;`. The right pane shows the output of the query, which is a table with three columns: `emp_name`, `salary`, and `salary_category`. The output table has five rows, each representing an employee and their salary category.

emp_name	salary	salary_category
Vivek Patel	62000	Low Salary
Sneha Jain	65000	Low Salary
Karan Mehta	91000	High Salary
Simran Kaur	70000	Medium Salary
Aditya Rao	60000	Low Salary

Case expression: Classify employees according to salary

SELECT emp_name,

salary, CASE

WHEN salary >= 75000 THEN 'High Salary' WHEN salary >= 60000 THEN 'Medium Salary' ELSE 'Low Salary'

END AS salary_category FROM Employee;

byteXL

queries.sql

```
52 (118, 'Karan Mehta', 91000, 32, 2),
53 (111, 'Sriran Kaur', 70000, 27, 2),
54 (112, 'Aditya Rao', 60000, 25, 2),
55 (113, 'Pooja Singh', 45000, 24, 3),
56 (114, 'Nitin Kumar', 52000, 28, 3),
57 (115, 'Riya Sharma', 58000, 29, 3),
58 (116, 'Mohit Verma', 63000, 31, 3),
59 (117, 'Kavita Gupta', 55000, 26, 3),
60 (118, 'Sahil Khan', 48000, 25, 2),
61 (119, 'Deepak Agarwal', 78000, 36, 4),
62 (120, 'Megha Joshi', 85000, 29, 4),
63 (121, 'Varun Malhotra', 95000, 34, 4),
64 (122, 'Isha Sinha', 67000, 27, 4),
65 (123, 'Manish Tiwari', 73000, 31, 4),
66 (124, 'Shreya Roy', 88000, 32, 4),
67 (125, 'Akash Gupta', 52000, 24, 5),
68 (126, 'Nisha Patel', 61000, 26, 5),
69 (127, 'Yash Sharma', 69000, 28, 5),
70 (128, 'Tanya Verma', 74000, 29, 5),
71 (129, 'Harsh Singh', 57000, 25, 5),
72 (130, 'Aditi Mehra', 80000, 30, 5);
73 -- =====
74 -- 6. INSERT 8 PROJECTS
75 -- =====
76 INSERT INTO Project VALUES
77 (201, 'E-Commerce Platform', 500000, 1),
78 (202, 'Cyber Security System', 750000, 1),
79 (203, 'AI Chatbot', 900000, 2),
80 (204, 'Machine Learning Model', 850000, 2),
81 (205, 'Employee Management System', 400000, 3),
82 (206, 'Financial Analysis Tool', 650000, 4),
83 (207, 'Marketing Automation', 550000, 5),
84 (208, 'Customer Analytics', 700000, 5);
85 SELECT emp_name,
86 salary, CASE
87 WHEN salary >= 75000 THEN 'High Salary' WHEN salary >= 60000 THEN 'Medium Salary' ELSE 'Low Salary'
88 END AS salary_category FROM Employee;
```

MySQL Run

STON

Input for the program (optional)

Output:

emp_name	salary	salary_category
Rahul Sharma	55000.00	Low Salary
Aman Verma	62000.00	Medium Salary
Neha Gupta	58000.00	Low Salary
Rohit Singh	72000.00	Medium Salary
Priya Mishra	68000.00	Medium Salary
Arjun Yadav	49000.00	Low Salary
Ananya Kapoor	75000.00	High Salary
Vivek Patel	82000.00	High Salary
Sneha Jain	65000.00	Medium Salary
Karan Mehta	91000.00	High Salary
Sriran Kaur	70000.00	Medium Salary
Aditya Rao	60000.00	Medium Salary
Pooja Singh	45000.00	Low Salary
Nitin Kumar	52000.00	Low Salary
Riya Sharma	58000.00	Low Salary
Rohit Verma	63000.00	Medium Salary
Kavita Gupta	55000.00	Low Salary
Sahil Khan	48000.00	Low Salary
Deepak Agarwal	78000.00	High Salary
Megha Joshi	85000.00	High Salary
Varun Malhotra	95000.00	High Salary
Isha Sinha	67000.00	Medium Salary
Manish Tiwari	73000.00	Medium Salary
Shreya Roy	88000.00	High Salary
Akash Gupta	52000.00	Low Salary
Nisha Patel	61000.00	Medium Salary
Yash Sharma	69000.00	Medium Salary
Tanya Verma	74000.00	Medium Salary
Harsh Singh	57000.00	Low Salary
Aditi Mehra	80000.00	Medium Salary

Light

Feedback

New History Save

Order By: Display employee in descending order of salary

SELECT emp_name, salary

FROM Employee

ORDER BY salary DESC;

byteXL

queries.sql

```
51 (109, 'Sneha Jain', 65000, 28, 2),
52 (110, 'Karan Mehta', 91000, 32, 2),
53 (111, 'Sriran Kaur', 70000, 27, 2),
54 (112, 'Aditya Rao', 60000, 25, 2),
55 (113, 'Pooja Singh', 45000, 24, 3),
56 (114, 'Nitin Kumar', 52000, 28, 3),
57 (115, 'Riya Sharma', 58000, 29, 3),
58 (116, 'Mohit Verma', 63000, 31, 3),
59 (117, 'Kavita Gupta', 55000, 26, 3),
60 (118, 'Sahil Khan', 48000, 25, 2),
61 (119, 'Deepak Agarwal', 78000, 36, 4),
62 (120, 'Megha Joshi', 85000, 29, 4),
63 (121, 'Varun Malhotra', 95000, 34, 4),
64 (122, 'Isha Sinha', 67000, 27, 4),
65 (123, 'Manish Tiwari', 73000, 31, 4),
66 (124, 'Shreya Roy', 88000, 32, 4),
67 (125, 'Akash Gupta', 52000, 24, 5),
68 (126, 'Nisha Patel', 61000, 26, 5),
69 (127, 'Yash Sharma', 69000, 28, 5),
70 (128, 'Tanya Verma', 74000, 29, 5),
71 (129, 'Harsh Singh', 57000, 25, 5),
72 (130, 'Aditi Mehra', 80000, 30, 5);
73 -- =====
74 -- 6. INSERT 8 PROJECTS
75 -- =====
76 INSERT INTO Project VALUES
77 (201, 'E-Commerce Platform', 500000, 1),
78 (202, 'Cyber Security System', 750000, 1),
79 (203, 'AI Chatbot', 900000, 2),
80 (204, 'Machine Learning Model', 850000, 2),
81 (205, 'Employee Management System', 400000, 3),
82 (206, 'Financial Analysis Tool', 650000, 4),
83 (207, 'Marketing Automation', 550000, 5),
84 (208, 'Customer Analytics', 700000, 5);
85 SELECT emp_name, salary
86 FROM Employee
87 ORDER BY salary DESC;
```

MySQL Run

STON

Input for the program (optional)

Output:

emp_name	salary
Varun Malhotra	95000.00
Karan Mehta	91000.00
Shreya Roy	88000.00
Megha Joshi	85000.00
Vivek Patel	82000.00
Aditi Mehra	80000.00
Deepak Agarwal	78000.00
Ananya Kapoor	75000.00
Tanya Verma	74000.00
Manish Tiwari	73000.00
Rohit Singh	72000.00
Sriran Kaur	70000.00
Yash Sharma	69000.00
Priya Mishra	68000.00
Isha Sinha	67000.00
Sneha Jain	65000.00
Rohit Verma	63000.00
Aman Verma	62000.00
Nisha Patel	61000.00
Aditya Rao	60000.00
Riya Sharma	58000.00
Neha Gupta	58000.00
Harsh Singh	57000.00
Kavita Gupta	55000.00
Rahul Sharma	55000.00
Nitin Kumar	52000.00
Akash Gupta	52000.00
Sahil Khan	48000.00
Pooja Singh	45000.00

Light

Feedback

New History Save

Experiment 4

Aim: Using the Employee schema, write queries with INNER JOIN, LEFT JOIN, self-join, 3-way join, correlated subqueries, EXISTS, and simulated INTERSECT and EXCEPT. Compare execution plans using EXPLAIN.

Source Code:

```
-- -----  
-- 1. CREATE TABLES  
-- -----  
  
CREATE TABLE Department (  
    dept_id INT PRIMARY KEY,  
    dept_name VARCHAR(50),  
    location VARCHAR(50)  
);  
  
CREATE TABLE Employee (  
    emp_id INT PRIMARY KEY,  
    emp_name VARCHAR(50),  
    salary DECIMAL(10,2),  
    job_title VARCHAR(50),  
    dept_id INT,  
    manager_id INT NULL,  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id),  
    FOREIGN KEY (manager_id) REFERENCES Employee(emp_id)  
);  
  
CREATE TABLE Project (  
    project_id INT PRIMARY KEY,  
    project_name VARCHAR(100),  
    dept_id INT,  
    budget DECIMAL(12,2),  
    FOREIGN KEY (dept_id) REFERENCES Department(dept_id)  
);  
  
CREATE TABLE Employee_Project (  
    emp_id INT,  
    project_id INT,  
    hours_worked INT,
```

```

PRIMARY KEY (emp_id, project_id),
FOREIGN KEY (emp_id) REFERENCES Employee(emp_id),
FOREIGN KEY (project_id) REFERENCES Project(project_id)
);

-- =====
-- 2. INSERT DATA
-- =====

INSERT INTO Department VALUES
(1, 'IT', 'Delhi'),
(2, 'HR', 'Mumbai'),
(3, 'Finance', 'Bangalore'),
(4, 'Marketing', 'Pune'),
(5, 'AI/ML', 'Hyderabad');

INSERT INTO Employee VALUES
(101, 'Rahul', 75000, 'Developer', 1, NULL),
(102, 'Aman', 65000, 'Developer', 1, 101),
(103, 'Priya', 90000, 'Manager', 2, NULL),
(104, 'Neha', 60000, 'HR Executive', 2, 103),
(105, 'Rohit', 85000, 'Analyst', 3, NULL),
(106, 'Sneha', 55000, 'Accountant', 3, 105),
(107, 'Karan', 70000, 'Marketing Executive', 4, NULL),
(108, 'Pooja', 50000, 'Marketing Executive', 4, 107),
(109, 'Arjun', 95000, 'AI Engineer', 5, NULL),
(110, 'Simran', 80000, 'ML Engineer', 5, 109),
(111, 'Vikas', 62000, 'Developer', 1, 101),
(112, 'Anjali', 58000, 'Developer', 1, 101),
(113, 'Mohit', 72000, 'HR Executive', 2, 103),
(114, 'Riya', 68000, 'Analyst', 3, 105),
(115, 'Varun', 78000, 'AI Engineer', 5, 109),
(116, 'Tanya', 52000, 'Marketing Executive', 4, 107),
(117, 'Aditya', 88000, 'Developer', 1, 101),
(118, 'Nisha', 61000, 'Accountant', 3, 105),
(119, 'Yash', 73000, 'ML Engineer', 5, 109),
(120, 'Kriti', 57000, 'HR Executive', 2, 103),

```

(121, 'Dev', 67000, 'Developer', 1, 101),
(122, 'Isha', 54000, 'Marketing Executive', 4, 107),
(123, 'Manish', 91000, 'AI Engineer', 5, 109),
(124, 'Ayesha', 63000, 'Analyst', 3, 105),
(125, 'Kabir', 76000, 'Developer', 1, 101),
(126, 'Meera', 59000, 'HR Executive', 2, 103),
(127, 'Sahil', 69000, 'Accountant', 3, 105),
(128, 'Diya', 81000, 'ML Engineer', 5, 109),
(129, 'Harsh', 71000, 'Marketing Executive', 4, 107),
(130, 'Muskan', 56000, 'HR Executive', 2, 103);

INSERT INTO Project VALUES

(201, 'Website Development', 1, 500000),
(202, 'Mobile App', 1, 700000),
(203, 'Recruitment System', 2, 300000),
(204, 'Financial Analysis', 3, 600000),
(205, 'Advertisement Campaign', 4, 400000),
(206, 'AI Chatbot', 5, 900000),
(207, 'Machine Learning Model', 5, 1000000),
(208, 'Data Analytics', 3, 800000);

INSERT INTO Employee_Project VALUES

(101,201,120),
(102,201,100),
(111,202,90),
(112,202,80),
(117,201,110),
(125,202,100),
(103,203,70),
(104,203,90),
(113,203,80),
(120,203,60),
(105,204,100),
(106,204,90),
(114,208,110),
(118,208,80),

(124,204,70),
 (127,208,90),
 (107,205,100),
 (108,205,80),
 (116,205,70),
 (122,205,60),
 (129,205,90),
 (109,206,120),
 (110,206,100),
 (115,207,110),
 (119,207,100),
 (123,206,90),
 (128,207,120);
 SELECT * FROM Employee;

The screenshot shows the byteXL SQL editor interface. The left sidebar contains navigation links: Home, Dashboard, Feedback Requests, Reports, Student Reports, Learning, AI Mentor, AI Interviewer, Learn, Editor (highlighted), Lab, Assessment, and Nimbus. The main editor area displays a list of SQL statements, including 11 INSERT INTO Employee_Project statements and a final SELECT * FROM Employee; statement. The right panel shows the output of the query, which is a table with 5 columns: emp_id, emp_name, salary, job_title, and dept. The output table contains 18 rows of data.

emp_id	emp_name	salary	job_title	dept
101	Rahul	75000.00	Developer	
102	Aman	65000.00	Developer	
103	Priya	90000.00	Manager	
104	Neha	60000.00	HR Executive	
105	Rohit	85000.00	Analyst	
106	Sneha	55000.00	Accountant	
107	Karan	70000.00	Marketing Executive	
108	Pooja	50000.00	Marketing Executive	
109	Arjun	95000.00	AI Engineer	
110	Samaran	80000.00	ML Engineer	
111	Vikas	62000.00	Developer	
112	Anjali	58000.00	Developer	
113	Mohit	72000.00	HR Executive	
114	Riya	68000.00	Analyst	
115	Varun	78000.00	AI Engineer	
116	Tanya	52000.00	Marketing Executive	
117	Aditya	88000.00	Developer	
118	Nisha	61000.00	Accountant	

OUTPUT:

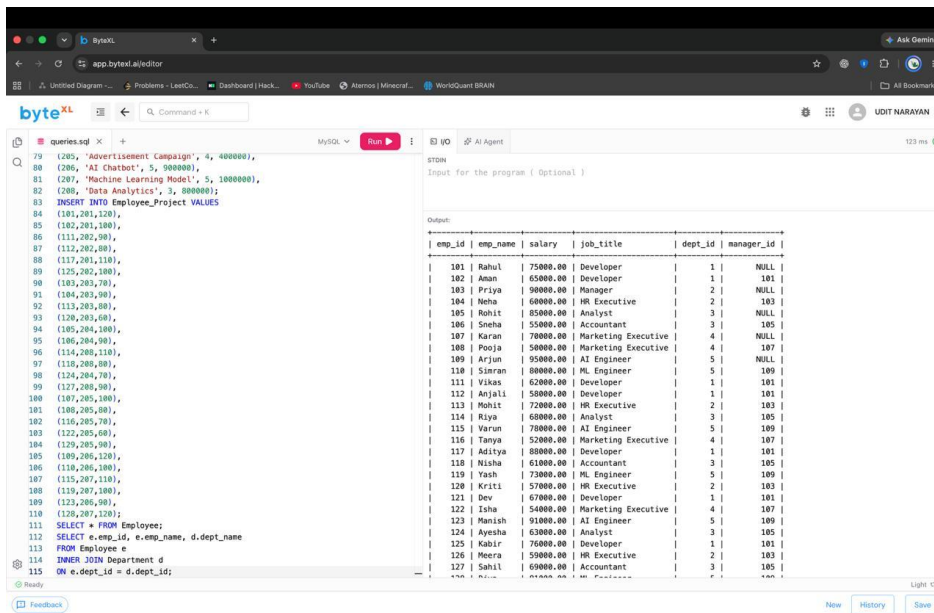
INNER JOIN:

SELECT e.emp_id, e.emp_name, d.dept_name

FROM Employee e

INNER JOIN Department d

ON e.dept_id = d.dept_id;



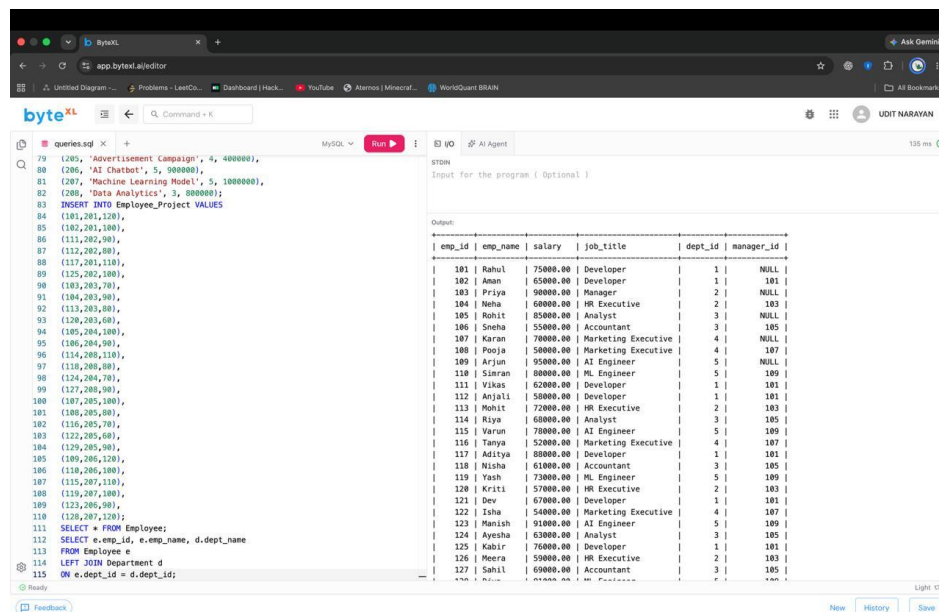
LEFT JOIN:

SELECT e.emp_id, e.emp_name, d.dept_name

FROM Employee e

LEFT JOIN Department d

ON e.dept_id = d.dept_id;



SELF JOIN:

SELECT

e.emp_name AS Employee, m.emp_name AS Manager

FROM Employee e

LEFT JOIN Employee m

ON e.manager_id = m.emp_id;

```

80 (200, 'AI Chatbot', 5, 900000);
81 (207, 'Machine Learning Model', 5, 1000000);
82 (208, 'Data Analytics', 3, 800000);
83 INSERT INTO Employee_Project VALUES
84 (101,201,120),
85 (102,201,100),
86 (111,202,90),
87 (112,202,80),
88 (117,201,110),
89 (125,202,100),
90 (103,203,70),
91 (104,203,90),
92 (113,203,80),
93 (120,203,60),
94 (105,204,100),
95 (106,204,90),
96 (114,200,110),
97 (118,200,80),
98 (124,204,70),
99 (127,200,90),
100 (107,205,100),
101 (108,205,80),
102 (116,205,70),
103 (122,205,60),
104 (129,205,90),
105 (109,206,120),
106 (110,206,100),
107 (115,207,110),
108 (119,207,100),
109 (123,206,90),
110 (120,207,120);
111 SELECT * FROM Employee;
112 SELECT
113 e.emp_name AS Employee, m.emp_name AS Manager
114 FROM Employee e
115 LEFT JOIN Employee m
116 ON e.manager_id = m.emp_id;

```

emp_id	emp_name	salary	job_title	dept_id	manager_id
101	Rahul	75000.00	Developer	1	NULL
102	Anan	65000.00	Developer	1	101
103	Priya	90000.00	Manager	2	NULL
104	Neha	60000.00	HR Executive	2	103
105	Rohit	85000.00	Analyst	3	NULL
106	Sneha	55000.00	Accountant	3	105
107	Karan	70000.00	Marketing Executive	4	NULL
108	Pooja	50000.00	Marketing Executive	4	107
109	Arjun	95000.00	AI Engineer	5	NULL
110	Sirran	80000.00	ML Engineer	5	109
111	Vikas	62000.00	Developer	1	101
112	Anjali	58000.00	Developer	1	101
113	Rohit	72000.00	HR Executive	2	103
114	Riya	68000.00	Analyst	3	105
115	Varun	78000.00	AI Engineer	5	109
116	Tanya	52000.00	Marketing Executive	4	107
117	Aditya	88000.00	Developer	1	101
118	Nisha	61000.00	Accountant	3	105
119	Yash	73000.00	ML Engineer	5	109
120	Kriti	57000.00	HR Executive	2	103
121	Dev	67000.00	Developer	1	101
122	Isha	54000.00	Marketing Executive	4	107
123	Manish	91000.00	AI Engineer	5	109
124	Ayesha	63000.00	Analyst	3	105
125	Kabir	76000.00	Developer	1	101
126	Meera	59000.00	HR Executive	2	103
127	Sahil	89000.00	Accountant	3	105

THREE-WAY JOIN:

SELECT e.emp_name, d.dept_name, p.project_name, ep.hours_worked

FROM Employee e

INNER JOIN Department d

ON e.dept_id = d.dept_id

INNER JOIN Employee_Project ep

ON e.emp_id = ep.emp_id

INNER JOIN Project p

ON ep.project_id = p.project_id;

```

84 (101,201,120),
85 (102,201,100),
86 (111,202,90),
87 (112,202,80),
88 (117,201,110),
89 (125,202,100),
90 (103,203,70),
91 (104,203,90),
92 (113,203,80),
93 (120,203,60),
94 (105,204,100),
95 (106,204,90),
96 (114,200,110),
97 (118,200,80),
98 (124,204,70),
99 (127,200,90),
100 (107,205,100),
101 (108,205,80),
102 (116,205,70),
103 (122,205,60),
104 (129,205,90),
105 (109,206,120),
106 (110,206,100),
107 (115,207,110),
108 (119,207,100),
109 (123,206,90),
110 (120,207,120);
111 SELECT * FROM Employee;
112 SELECT
113 e.emp_name, d.dept_name, p.project_name, ep.hours_worked
114 FROM Employee e
115 INNER JOIN Department d
116 ON e.dept_id = d.dept_id
117 INNER JOIN Employee_Project ep
118 ON e.emp_id = ep.emp_id
119 INNER JOIN Project p
120 ON ep.project_id = p.project_id;

```

emp_id	emp_name	salary	job_title	dept_id	manager_id
101	Rahul	75000.00	Developer	1	NULL
102	Anan	65000.00	Developer	1	101
103	Priya	90000.00	Manager	2	NULL
104	Neha	60000.00	HR Executive	2	103
105	Rohit	85000.00	Analyst	3	NULL
106	Sneha	55000.00	Accountant	3	105
107	Karan	70000.00	Marketing Executive	4	NULL
108	Pooja	50000.00	Marketing Executive	4	107
109	Arjun	95000.00	AI Engineer	5	NULL
110	Sirran	80000.00	ML Engineer	5	109
111	Vikas	62000.00	Developer	1	101
112	Anjali	58000.00	Developer	1	101
113	Rohit	72000.00	HR Executive	2	103
114	Riya	68000.00	Analyst	3	105
115	Varun	78000.00	AI Engineer	5	109
116	Tanya	52000.00	Marketing Executive	4	107
117	Aditya	88000.00	Developer	1	101
118	Nisha	61000.00	Accountant	3	105
119	Yash	73000.00	ML Engineer	5	109
120	Kriti	57000.00	HR Executive	2	103
121	Dev	67000.00	Developer	1	101
122	Isha	54000.00	Marketing Executive	4	107
123	Manish	91000.00	AI Engineer	5	109
124	Ayesha	63000.00	Analyst	3	105
125	Kabir	76000.00	Developer	1	101
126	Meera	59000.00	HR Executive	2	103
127	Sahil	89000.00	Accountant	3	105

CORRELATED SUBQUERY:

```
SELECT e.emp_id, e.emp_name, e.salary, e.dept_id
```

```
FROM Employee e
```

```
WHERE e.salary > ( SELECT AVG(e2.salary)
```

```
FROM Employee e2
```

```
WHERE e2.dept_id = e.dept_id );
```

The screenshot shows the byteXL SQL editor with a query window on the left and an output window on the right. The query is a correlated subquery that filters employees based on their salary being greater than the average salary of their department.

```
78 (200, 'AI Chatbot', 3, 8000000),
81 (207, 'Machine Learning Model', 5, 10000000),
82 (208, 'Data Analytics', 3, 8000000);
83 INSERT INTO Employee_Project VALUES
84 (181,201,120),
85 (182,201,100),
86 (111,202,90),
87 (112,202,200),
88 (117,201,110),
89 (125,202,100),
90 (103,203,70),
91 (104,203,90),
92 (113,203,80),
93 (120,203,60),
94 (105,204,100),
95 (106,204,90),
96 (114,200,110),
97 (110,200,80),
98 (124,204,70),
99 (127,200,90),
100 (107,205,100),
101 (100,205,80),
102 (115,205,70),
103 (122,205,60),
104 (129,205,90),
105 (109,206,120),
106 (110,206,100),
107 (115,207,110),
108 (119,207,100),
109 (123,206,90),
110 (120,207,120);
111 SELECT * FROM Employee;
112 SELECT e.emp_id, e.emp_name, e.salary, e.dept_id
113 FROM Employee e
114 WHERE e.salary > ( SELECT AVG(e2.salary)
115 FROM Employee e2
116 WHERE e2.dept_id = e.dept_id );
```

The output window displays a table with 6 columns: emp_id, emp_name, salary, job_title, dept_id, and manager_id. It lists 27 employees with their respective details.

emp_id	emp_name	salary	job_title	dept_id	manager_id
101	Rahul	75000.00	Developer	1	NULL
102	Anan	65000.00	Developer	1	101
103	Priya	90000.00	Manager	2	NULL
104	Neha	60000.00	HR Executive	2	103
105	Rohit	85000.00	Analyst	3	NULL
106	Sneha	55000.00	Accountant	3	105
107	Karan	70000.00	Marketing Executive	4	NULL
108	Pooja	50000.00	Marketing Executive	4	107
109	Arjun	95000.00	AI Engineer	5	NULL
110	Sirran	80000.00	ML Engineer	5	109
111	Vikas	62000.00	Developer	1	101
112	Anjali	50000.00	Developer	1	101
113	Mohit	72000.00	HR Executive	2	103
114	Riya	60000.00	Analyst	3	105
115	Varun	70000.00	AI Engineer	5	109
116	Tanya	52000.00	Marketing Executive	4	107
117	Aditya	80000.00	Developer	1	101
118	Nisha	61000.00	Accountant	3	105
119	Yash	73000.00	ML Engineer	5	109
120	Kriti	57000.00	HR Executive	2	103
121	Dev	67000.00	Developer	1	101
122	Isha	54000.00	Marketing Executive	4	107
123	Manish	91000.00	AI Engineer	5	109
124	Ayesha	63000.00	Analyst	3	105
125	Kabir	70000.00	Developer	1	101
126	Meera	50000.00	HR Executive	2	103
127	Sahil	69000.00	Accountant	3	105

EXISTS:

```
SELECT e.emp_id, e.emp_name
```

```
FROM Employee e
```

```
WHERE EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```

The screenshot shows the byteXL SQL editor with a query window on the left and an output window on the right. The query uses the EXISTS operator to filter employees who have a corresponding record in the Employee_Project table.

```
78 'Financial Analysis', 3, 8000000),
79 'Advertisement Campaign', 4, 4000000),
80 'AI Chatbot', 3, 8000000),
81 'Machine Learning Model', 5, 10000000);
82 INSERT INTO Employee_Project VALUES
83 (11,201,120),
84 (12,201,100),
85 (1,202,90),
86 (2,202,200),
87 (7,201,110),
88 (5,202,100),
89 (3,203,70),
90 (4,203,90),
91 (6,203,80),
92 (3,203,60),
93 (6,204,100),
94 (6,204,90),
95 (4,200,110),
96 (4,200,80),
97 (6,200,70),
98 (6,200,90),
99 (7,200,90),
100 (7,205,100),
101 (6,205,80),
102 (6,205,70),
103 (6,205,60),
104 (6,205,90),
105 (6,206,120),
106 (6,206,100),
107 (5,207,110),
108 (6,207,100),
109 (6,206,90),
110 (6,207,120);
111 SELECT * FROM Employee;
112 SELECT e.emp_id, e.emp_name
113 FROM Employee e
114 WHERE EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```

The output window displays a table with 6 columns: emp_id, emp_name, salary, job_title, dept_id, and manager_id. It lists 27 employees, which is the same set as the previous screenshot.

emp_id	emp_name	salary	job_title	dept_id	manager_id
101	Rahul	75000.00	Developer	1	NULL
102	Anan	65000.00	Developer	1	101
103	Priya	90000.00	Manager	2	NULL
104	Neha	60000.00	HR Executive	2	103
105	Rohit	85000.00	Analyst	3	NULL
106	Sneha	55000.00	Accountant	3	105
107	Karan	70000.00	Marketing Executive	4	NULL
108	Pooja	50000.00	Marketing Executive	4	107
109	Arjun	95000.00	AI Engineer	5	NULL
110	Sirran	80000.00	ML Engineer	5	109
111	Vikas	62000.00	Developer	1	101
112	Anjali	50000.00	Developer	1	101
113	Mohit	72000.00	HR Executive	2	103
114	Riya	60000.00	Analyst	3	105
115	Varun	70000.00	AI Engineer	5	109
116	Tanya	52000.00	Marketing Executive	4	107
117	Aditya	80000.00	Developer	1	101
118	Nisha	61000.00	Accountant	3	105
119	Yash	73000.00	ML Engineer	5	109
120	Kriti	57000.00	HR Executive	2	103
121	Dev	67000.00	Developer	1	101
122	Isha	54000.00	Marketing Executive	4	107
123	Manish	91000.00	AI Engineer	5	109
124	Ayesha	63000.00	Analyst	3	105
125	Kabir	70000.00	Developer	1	101
126	Meera	50000.00	HR Executive	2	103
127	Sahil	69000.00	Accountant	3	105
128	Diya	81000.00	ML Engineer	5	109
129	Harsh	71000.00	Marketing Executive	4	107
130	Muskan	56000.00	HR Executive	2	103

SIMULATED INTERSECT:

```
SELECT e.emp_id, e.emp_name
```

```
FROM Employee e
```

```
WHERE e.salary > 70000
```

```
AND EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```

The screenshot shows the byteXL SQL editor interface. The left pane displays a SQL query with line numbers 79 to 115. The query is as follows:

```
79 (205, 'Advertisement Campaign', 4, 400000),
80 (206, 'AI Chatbot', 5, 900000),
81 (207, 'Machine Learning Model', 5, 1000000),
82 (208, 'Data Analytics', 3, 800000);
83 INSERT INTO Employee_Project VALUES
84 (101,201,120),
85 (102,201,100),
86 (111,202,90),
87 (112,202,80),
88 (117,201,110),
89 (125,202,100),
90 (103,203,70),
91 (104,203,90),
92 (113,203,80),
93 (120,203,60),
94 (105,204,100),
95 (106,204,90),
96 (114,200,110),
97 (110,200,80),
98 (124,204,70),
99 (127,200,90),
100 (107,205,100),
101 (100,205,80),
102 (116,205,70),
103 (122,205,60),
104 (129,205,90),
105 (109,206,120),
106 (110,206,100),
107 (115,207,110),
108 (119,207,100),
109 (123,206,90),
110 (120,207,120);
111 SELECT * FROM Employee;
112 SELECT e.emp_id, e.emp_name
113 FROM Employee e
114 WHERE e.salary > 70000
115 AND EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```

The right pane shows the output of the query, which is a table with 6 columns: emp_id, emp_name, salary, job_title, dept_id, and manager_id. The table contains 130 rows of data.

emp_id	emp_name	salary	job_title	dept_id	manager_id
101	Rahul	75000.00	Developer	1	NULL
102	Aman	65000.00	Developer	1	101
103	Priya	90000.00	Manager	2	NULL
104	Neha	60000.00	HR Executive	2	103
105	Rohit	85000.00	Analyst	3	NULL
106	Sneha	55000.00	Accountant	3	105
107	Karan	70000.00	Marketing Executive	4	NULL
108	Pooja	50000.00	Marketing Executive	4	107
109	Arjun	95000.00	AI Engineer	5	NULL
110	Sirran	80000.00	ML Engineer	5	109
111	Vikas	62000.00	Developer	1	101
112	Anjali	50000.00	Developer	1	101
113	Rohit	72000.00	HR Executive	2	103
114	Riya	68000.00	Analyst	3	105
115	Varun	78000.00	AI Engineer	5	109
116	Tanya	52000.00	Marketing Executive	4	107
117	Aditya	88000.00	Developer	1	101
118	Nisha	61000.00	Accountant	3	105
119	Yash	73000.00	ML Engineer	5	109
120	Kriti	57000.00	HR Executive	2	103
121	Dev	67000.00	Developer	1	101
122	Isha	54000.00	Marketing Executive	4	107
123	Manish	91000.00	AI Engineer	5	109
124	Ayesha	63000.00	Analyst	3	105
125	Kabir	76000.00	Developer	1	101
126	Meera	59000.00	HR Executive	2	103
127	Sahil	69000.00	Accountant	3	105
128	Diya	81000.00	ML Engineer	5	109
129	Harsh	71000.00	Marketing Executive	4	107
130	Muskan	56000.00	HR Executive	2	103

SIMULATED EXCEPT:

```
SELECT e.emp_id, e.emp_name
```

```
FROM Employee e
```

```
WHERE NOT EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```

The screenshot shows the byteXL SQL editor interface. The left pane displays a SQL query with line numbers 78 to 113. The query is as follows:

```
78 (204, 'Financial Analysis', 3, 600000),
79 (205, 'Advertisement Campaign', 4, 400000),
80 (206, 'AI Chatbot', 5, 900000),
81 (207, 'Machine Learning Model', 5, 1000000),
82 (208, 'Data Analytics', 3, 800000);
83 INSERT INTO Employee_Project VALUES
84 (101,201,120),
85 (102,201,100),
86 (111,202,90),
87 (112,202,80),
88 (117,201,110),
89 (125,202,100),
90 (103,203,70),
91 (104,203,90),
92 (113,203,80),
93 (120,203,60),
94 (105,204,100),
95 (106,204,90),
96 (114,200,110),
97 (110,200,80),
98 (124,204,70),
99 (127,200,90),
100 (107,205,100),
101 (100,205,80),
102 (116,205,70),
103 (122,205,60),
104 (129,205,90),
105 (109,206,120),
106 (110,206,100),
107 (115,207,110),
108 (119,207,100),
109 (123,206,90),
110 (120,207,120);
111 SELECT * FROM Employee;
112 SELECT e.emp_id, e.emp_name
113 FROM Employee e
114 WHERE NOT EXISTS ( SELECT 1 FROM Employee_Project ep WHERE ep.emp_id = e.emp_id );
```

The right pane shows the output of the query, which is a table with 6 columns: emp_id, emp_name, salary, job_title, dept_id, and manager_id. The table contains 130 rows of data.

emp_id	emp_name	salary	job_title	dept_id	manager_id
101	Rahul	75000.00	Developer	1	NULL
102	Aman	65000.00	Developer	1	101
103	Priya	90000.00	Manager	2	NULL
104	Neha	60000.00	HR Executive	2	103
105	Rohit	85000.00	Analyst	3	NULL
106	Sneha	55000.00	Accountant	3	105
107	Karan	70000.00	Marketing Executive	4	NULL
108	Pooja	50000.00	Marketing Executive	4	107
109	Arjun	95000.00	AI Engineer	5	NULL
110	Sirran	80000.00	ML Engineer	5	109
111	Vikas	62000.00	Developer	1	101
112	Anjali	50000.00	Developer	1	101
113	Rohit	72000.00	HR Executive	2	103
114	Riya	68000.00	Analyst	3	105
115	Varun	78000.00	AI Engineer	5	109
116	Tanya	52000.00	Marketing Executive	4	107
117	Aditya	88000.00	Developer	1	101
118	Nisha	61000.00	Accountant	3	105
119	Yash	73000.00	ML Engineer	5	109
120	Kriti	57000.00	HR Executive	2	103
121	Dev	67000.00	Developer	1	101
122	Isha	54000.00	Marketing Executive	4	107
123	Manish	91000.00	AI Engineer	5	109
124	Ayesha	63000.00	Analyst	3	105
125	Kabir	76000.00	Developer	1	101
126	Meera	59000.00	HR Executive	2	103
127	Sahil	69000.00	Accountant	3	105
128	Diya	81000.00	ML Engineer	5	109
129	Harsh	71000.00	Marketing Executive	4	107
130	Muskan	56000.00	HR Executive	2	103

EXPLAIN:

EXPLAIN

SELECT e.emp_name, d.dept_name

FROM Employee e

INNER JOIN Department d

ON e.dept_id = d.dept_id;

The screenshot shows the byteXL SQL editor interface. The left pane displays a SQL query with line numbers 80 to 116. The query is as follows:

```
80 (200, 'AI Chatbot', 5, 9000000),
81 (207, 'Machine Learning Model', 5, 10000000),
82 (208, 'Data Analytics', 3, 8000000);
83 INSERT INTO Employee_Project VALUES
84 (181,201,120),
85 (182,201,100),
86 (111,202,90),
87 (112,202,80),
88 (117,201,110),
89 (125,202,100),
90 (101,203,70),
91 (104,203,90),
92 (113,203,80),
93 (120,203,60),
94 (105,204,100),
95 (106,204,90),
96 (114,204,110),
97 (116,204,80),
98 (124,204,70),
99 (127,204,90),
100 (107,205,100),
101 (108,205,80),
102 (110,205,70),
103 (122,205,60),
104 (129,205,90),
105 (109,206,120),
106 (118,206,100),
107 (115,207,110),
108 (119,207,100),
109 (123,206,90),
110 (120,207,120);
111 SELECT * FROM Employee;
112 EXPLAIN
113 SELECT e.emp_name, d.dept_name
114 FROM Employee e
115 INNER JOIN Department d
116 ON e.dept_id = d.dept_id;
```

The right pane shows the output of the query, which is a table with 5 columns: emp_id, emp_name, salary, job_title, and dept_id. The table contains 29 rows of data. The bottom right corner of the interface shows the user's name, Udit Narayan, and the time taken for the query to execute, 104 ms.

emp_id	emp_name	salary	job_title	dept_id	manager_id
101	Rahul	75000.00	Developer	1	NULL
102	Aman	65000.00	Developer	1	101
103	Priya	90000.00	Manager	2	NULL
104	Neha	60000.00	HR Executive	2	103
105	Rohit	85000.00	Analyst	3	NULL
106	Sneha	55000.00	Accountant	3	105
107	Karan	70000.00	Marketing Executive	4	NULL
108	Pooja	50000.00	Marketing Executive	4	107
109	Arjun	95000.00	AI Engineer	5	NULL
110	Sirjan	80000.00	ML Engineer	5	109
111	Vikas	62000.00	Developer	1	101
112	Anjali	58000.00	Developer	1	101
113	Mohit	72000.00	HR Executive	2	103
114	Riya	68000.00	Analyst	3	105
115	Varun	78000.00	AI Engineer	5	109
116	Tanya	52000.00	Marketing Executive	4	107
117	Aditya	88000.00	Developer	1	101
118	Nisha	61000.00	Accountant	3	105
119	Yash	73000.00	ML Engineer	5	109
120	Kriti	57000.00	HR Executive	2	103
121	Dev	67000.00	Developer	1	101
122	Isha	54000.00	Marketing Executive	4	107
123	Harish	91000.00	AI Engineer	5	109
124	Ayesha	63000.00	Analyst	3	105
125	Kabir	76000.00	Developer	1	101
126	Heera	59000.00	HR Executive	2	103
127	Sahil	69000.00	Accountant	3	105
128	Diya	81000.00	ML Engineer	5	109
129	Harsh	71000.00	Marketing Executive	4	107

Experiment 5

Aim: To create SQL views for department salary summary and employee hierarchy, test the updatability of views, and implement a recursive CTE to display reporting chains.

Source Code:

```
-- =====
-- 1. CREATE DEPARTMENT TABLE
-- =====

CREATE TABLE department (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL
);

-- Insert departments
INSERT INTO department VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance');

-- =====
-- 2. CREATE EMPLOYEE TABLE
-- =====

CREATE TABLE employee (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(100) NOT NULL,
    department_id INT,
    manager_id INT NULL,
    salary DECIMAL(10,2),
    FOREIGN KEY (department_id)
        REFERENCES department(department_id),
    FOREIGN KEY (manager_id)
        REFERENCES employee(employee_id)
);

-- Insert employees
INSERT INTO employee VALUES
(101, 'Amit', 1, NULL, 90000),
(102, 'Neha', 1, 101, 65000),
```

```

(103, 'Rahul', 1, 101, 60000),
(104, 'Priya', 2, NULL, 80000),
(105, 'Karan', 2, 104, 55000),
(106, 'Riya', 3, NULL, 85000),
(107, 'Arjun', 3, 106, 50000);

-- =====

-- 3. VIEW FOR DEPARTMENT SALARY SUMMARY

-- =====

CREATE VIEW department_salary_summary AS
SELECT
    d.department_id,
    d.department_name,
    COUNT(e.employee_id) AS employee_count,
    SUM(e.salary) AS total_salary,
    AVG(e.salary) AS average_salary
FROM department d
LEFT JOIN employee e
    ON d.department_id = e.department_id
GROUP BY
    d.department_id,
    d.department_name;

-- Display the view

SELECT * FROM department_salary_summary;

-- =====

-- 4. VIEW FOR EMPLOYEE HIERARCHY

-- =====

CREATE VIEW employee_hierarchy AS
SELECT
    e.employee_id,
    e.employee_name,
    e.department_id,
    e.manager_id,
    m.employee_name AS manager_name,
    e.salary

```

```

FROM employee e
LEFT JOIN employee m
    ON e.manager_id = m.employee_id;
-- Display the view
SELECT * FROM employee_hierarchy;

-- =====

-- 5. TEST VIEW UPDATABILITY
-- =====

-- Simple single-table view
CREATE VIEW employee_basic AS
SELECT
    employee_id,
    employee_name,
    salary
FROM employee;
-- Update salary through the view
UPDATE employee_basic
SET salary = salary + 5000
WHERE employee_id = 102;
-- Check updated record
SELECT *
FROM employee_basic
WHERE employee_id = 102;
-- Check original table also
SELECT *
FROM employee
WHERE employee_id = 102;

-- =====

-- 6. RECURSIVE CTE FOR REPORTING CHAINS
-- =====

WITH RECURSIVE reporting_chain AS (
    -- Anchor member:
    -- Employees who have no manager
    SELECT

```

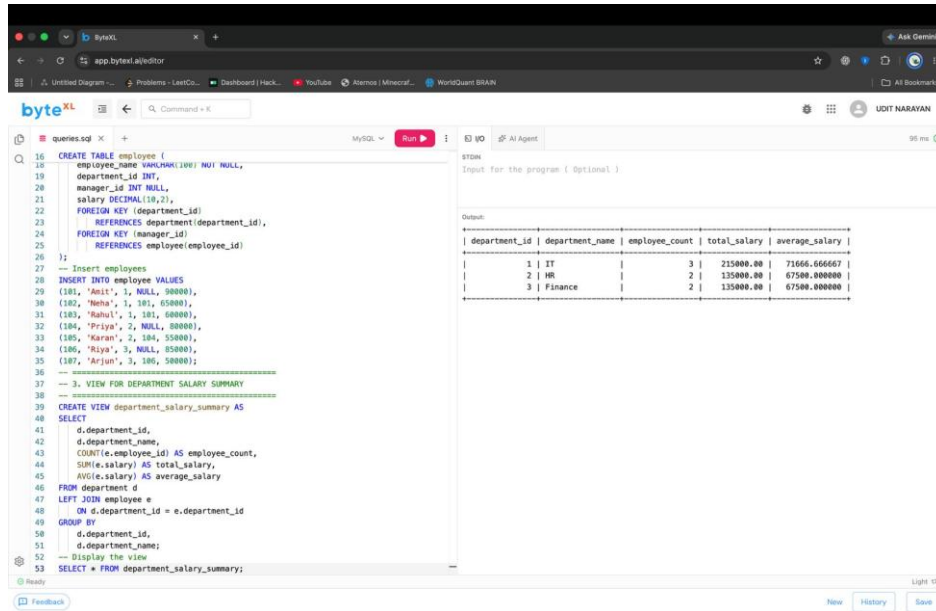
```

    employee_id,
    employee_name,
    manager_id,
    0 AS level,
    CAST(employee_name AS CHAR(1000)) AS reporting_path
FROM employee
WHERE manager_id IS NULL
UNION ALL
-- Recursive member:
-- Find employees reporting to each employee
SELECT
    e.employee_id,
    e.employee_name,
    e.manager_id,
    rc.level + 1,
    CONCAT(
        rc.reporting_path,
        ' -> ',
        e.employee_name
    )
FROM employee e
INNER JOIN reporting_chain rc
    ON e.manager_id = rc.employee_id
)
SELECT
    employee_id,
    employee_name,
    manager_id,
    level,
    reporting_path
FROM reporting_chain
ORDER BY reporting_path;

```

OUTPUT:

1: DEPARTMENT SALARY:



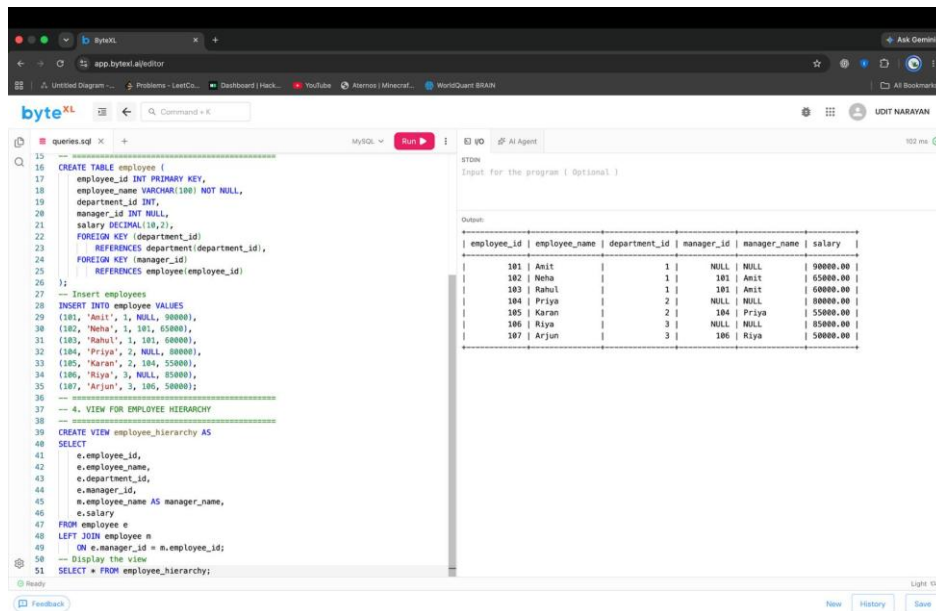
The screenshot shows the byteXL MySQL IDE interface. The left pane displays the SQL query editor with the following code:

```
16 CREATE TABLE employee (  
17   employee_name VARCHAR(100) NOT NULL,  
18   department_id INT,  
19   manager_id INT NULL,  
20   salary DECIMAL(10,2),  
21   FOREIGN KEY (department_id)  
22     REFERENCES department(department_id),  
23   FOREIGN KEY (manager_id)  
24     REFERENCES employee(employee_id)  
25 );  
26  
27 -- Insert employees  
28 INSERT INTO employee VALUES  
29 (101, 'Amit', 1, NULL, 90000),  
30 (102, 'Neha', 1, 101, 65000),  
31 (103, 'Rahul', 1, 101, 60000),  
32 (104, 'Priya', 2, NULL, 80000),  
33 (105, 'Karan', 2, 104, 55000),  
34 (106, 'Riya', 3, NULL, 85000),  
35 (107, 'Arjun', 3, 106, 50000);  
36  
37 -- 3. VIEW FOR DEPARTMENT SALARY SUMMARY  
38  
39 CREATE VIEW department_salary_summary AS  
40 SELECT  
41   d.department_id,  
42   d.department_name,  
43   COUNT(e.employee_id) AS employee_count,  
44   SUM(e.salary) AS total_salary,  
45   AVG(e.salary) AS average_salary  
46 FROM department d  
47 LEFT JOIN employee e  
48   ON d.department_id = e.department_id  
49 GROUP BY  
50   d.department_id,  
51   d.department_name;  
52  
53 -- Display the view  
54 SELECT * FROM department_salary_summary;
```

The right pane shows the output of the query, which is a table with 5 columns: department_id, department_name, employee_count, total_salary, and average_salary. The output is as follows:

department_id	department_name	employee_count	total_salary	average_salary
1	IT	3	215000.00	71666.666667
2	HR	2	135000.00	67500.000000
3	Finance	2	135000.00	67500.000000

2: EMPLOYEE HEIRARCHY:



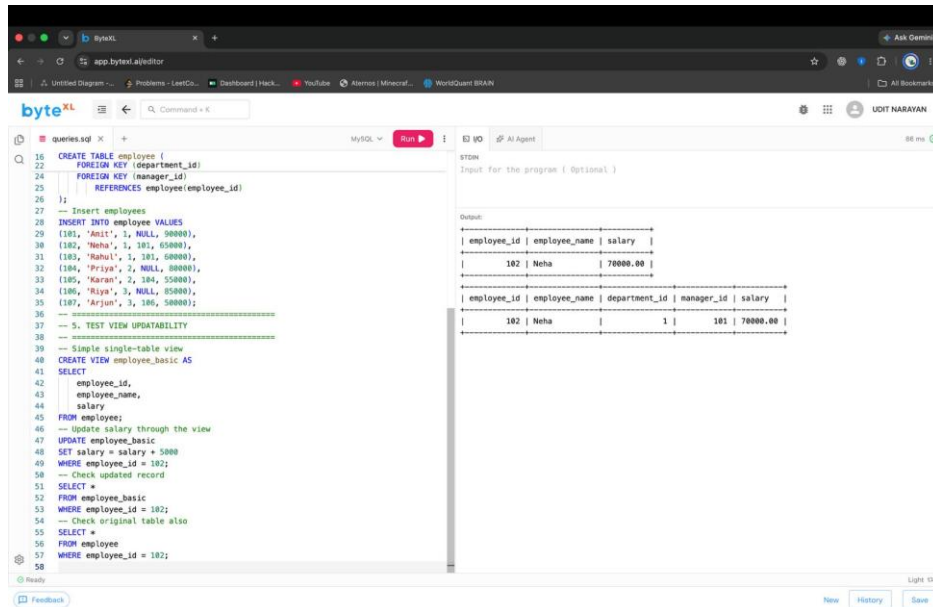
The screenshot shows the byteXL MySQL IDE interface. The left pane displays the SQL query editor with the following code:

```
16 CREATE TABLE employee (  
17   employee_id INT PRIMARY KEY,  
18   employee_name VARCHAR(100) NOT NULL,  
19   department_id INT,  
20   manager_id INT NULL,  
21   salary DECIMAL(10,2),  
22   FOREIGN KEY (department_id)  
23     REFERENCES department(department_id),  
24   FOREIGN KEY (manager_id)  
25     REFERENCES employee(employee_id)  
26 );  
27  
28 -- Insert employees  
29 INSERT INTO employee VALUES  
30 (101, 'Amit', 1, NULL, 90000),  
31 (102, 'Neha', 1, 101, 65000),  
32 (103, 'Rahul', 1, 101, 60000),  
33 (104, 'Priya', 2, NULL, 80000),  
34 (105, 'Karan', 2, 104, 55000),  
35 (106, 'Riya', 3, NULL, 85000),  
36 (107, 'Arjun', 3, 106, 50000);  
37  
38 -- 4. VIEW FOR EMPLOYEE HIERARCHY  
39  
40 CREATE VIEW employee_hierarchy AS  
41 SELECT  
42   e.employee_id,  
43   e.employee_name,  
44   e.department_id,  
45   m.employee_id AS manager_id,  
46   m.employee_name AS manager_name,  
47   e.salary  
48 FROM employee e  
49 LEFT JOIN employee m  
50   ON e.manager_id = m.employee_id;  
51  
52 -- Display the view  
53 SELECT * FROM employee_hierarchy;
```

The right pane shows the output of the query, which is a table with 6 columns: employee_id, employee_name, department_id, manager_id, manager_name, and salary. The output is as follows:

employee_id	employee_name	department_id	manager_id	manager_name	salary
101	Amit	1	NULL	NULL	90000.00
102	Neha	1	101	Amit	65000.00
103	Rahul	1	101	Amit	60000.00
104	Priya	2	NULL	NULL	80000.00
105	Karan	2	104	Priya	55000.00
106	Riya	3	NULL	NULL	85000.00
107	Arjun	3	106	Riya	50000.00

3: UPDATABILITY TEST:



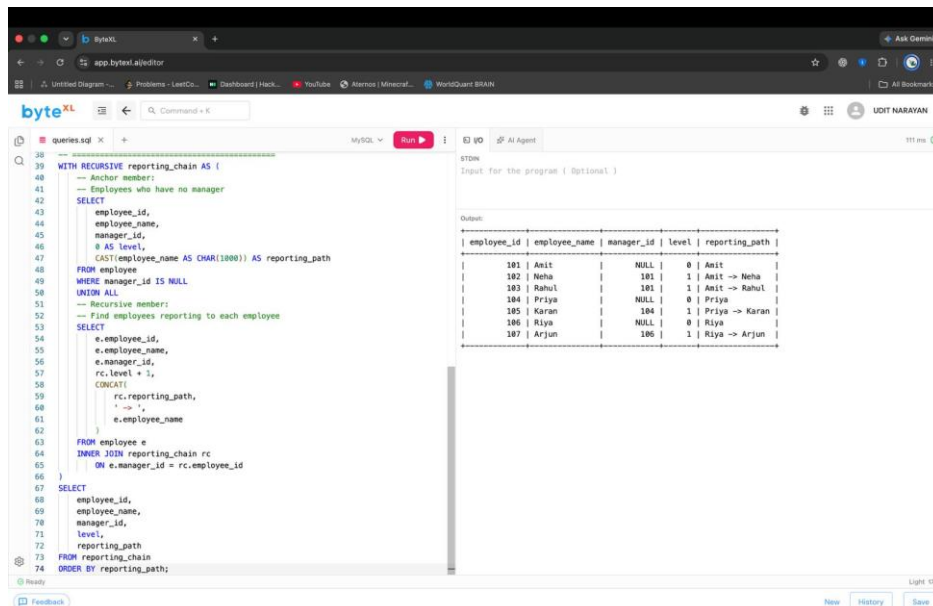
```
16 CREATE TABLE employee (
22   FOREIGN KEY (department_id)
24   FOREIGN KEY (manager_id)
25   REFERENCES employee(employee_id)
26 );
27 -- Insert employees
28 INSERT INTO employee VALUES
29 (101, 'Amit', 1, NULL, 90000),
30 (102, 'Neha', 1, 101, 60000),
31 (103, 'Rahul', 1, 101, 60000),
32 (104, 'Priya', 2, NULL, 80000),
33 (105, 'Karan', 2, 104, 55000),
34 (106, 'Riya', 3, NULL, 85000),
35 (107, 'Arjun', 3, 106, 50000);
36
37 -- 5. TEST VIEW UPDATABILITY
38 -- =====
39 -- Simple single-table view
40 CREATE VIEW employee_basic AS
41 SELECT
42   employee_id,
43   employee_name,
44   salary
45 FROM employee;
46 -- Update salary through the view
47 UPDATE employee_basic
48 SET salary = salary + 5000
49 WHERE employee_id = 102;
50 -- Check updated record
51 SELECT *
52 FROM employee_basic
53 WHERE employee_id = 102;
54 -- Check original table also
55 SELECT *
56 FROM employee
57 WHERE employee_id = 102;
58
```

Output:

employee_id	employee_name	salary
102	Neha	70000.00

employee_id	employee_name	department_id	manager_id	salary
102	Neha	1	101	70000.00

4: RECURSIVE CTE REPORTING CHAIN:



```
38
39 WITH RECURSIVE reporting_chain AS (
40   -- Anchor member:
41   -- Employees who have no manager
42   SELECT
43     employee_id,
44     employee_name,
45     manager_id,
46     0 AS level,
47     CAST(employee_name AS CHAR(1000)) AS reporting_path
48 FROM employee
49 WHERE manager_id IS NULL
50 UNION ALL
51 -- Recursive member:
52 -- Find employees reporting to each employee
53 SELECT
54   e.employee_id,
55   e.employee_name,
56   e.manager_id,
57   rc.level + 1,
58   CONCAT(
59     rc.reporting_path,
60     ' -> ',
61     e.employee_name
62   )
63 FROM employee e
64 INNER JOIN reporting_chain rc
65 ON e.manager_id = rc.employee_id
66 )
67 SELECT
68   employee_id,
69   employee_name,
70   manager_id,
71   level,
72   reporting_path
73 FROM reporting_chain
74 ORDER BY reporting_path;

```

Output:

employee_id	employee_name	manager_id	level	reporting_path
101	Amit	NULL	0	Amit
102	Neha	101	1	Amit -> Neha
103	Rahul	101	1	Amit -> Rahul
104	Priya	NULL	0	Priya
105	Karan	104	1	Priya -> Karan
106	Riya	NULL	0	Riya
107	Arjun	106	1	Riya -> Arjun

Experiment 6

Aim: To create a stored procedure transfer_employee(emp_id, new_dept_id) with validation and error handling, and to implement database triggers for salary validation and audit logging.

Source Code:

```
-- =====
-- 1. CREATE DATABASE TABLES
-- =====

CREATE TABLE department (
    department_id INT PRIMARY KEY,
    department_name VARCHAR(50) NOT NULL
);

CREATE TABLE employee (
    employee_id INT PRIMARY KEY,
    employee_name VARCHAR(100) NOT NULL,
    department_id INT,
    salary DECIMAL(10,2),
    FOREIGN KEY (department_id) REFERENCES department(department_id)
);

-- Sample departments
INSERT INTO department VALUES
(1, 'IT'),
(2, 'HR'),
(3, 'Finance'),
(4, 'Marketing');

-- Sample employees
INSERT INTO employee VALUES
(101, 'Rahul', 1, 50000),
(102, 'Aman', 2, 45000),
(103, 'Neha', 1, 60000),
(104, 'Priya', 3, 55000),
(105, 'Rohan', 4, 40000);

-- =====
-- 2. CREATE AUDIT TABLE
```

```

-- =====
CREATE TABLE employee_audit (
    audit_id INT AUTO_INCREMENT PRIMARY KEY,
    employee_id INT,
    old_department_id INT,
    new_department_id INT,
    old_salary DECIMAL(10,2),
    new_salary DECIMAL(10,2),
    action_type VARCHAR(50),
    action_time TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);
-- =====

-- 3. STORED PROCEDURE
-- transfer_employee(emp_id, new_dept_id)
-- =====

```

DELIMITER //

```

CREATE PROCEDURE transfer_employee(
    IN p_emp_id INT,
    IN p_new_dept_id INT
)
BEGIN
    DECLARE v_old_dept_id INT;
    DECLARE v_emp_count INT DEFAULT 0;
    DECLARE v_dept_count INT DEFAULT 0;
    -- Error handling
    DECLARE EXIT HANDLER FOR SQLEXCEPTION
    BEGIN
        ROLLBACK;
        SELECT 'Error: Transfer failed.' AS message;
    END;
    START TRANSACTION;
    -- Check employee exists

```

```

SELECT COUNT(*)
INTO v_emp_count
FROM employee
WHERE employee_id = p_emp_id;
IF v_emp_count = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error: Employee does not exist.';
END IF;
-- Check department exists
SELECT COUNT(*)
INTO v_dept_count
FROM department
WHERE department_id = p_new_dept_id;
IF v_dept_count = 0 THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error: New department does not exist.';
END IF;
-- Get current department
SELECT department_id
INTO v_old_dept_id
FROM employee
WHERE employee_id = p_emp_id;
-- Check if already in department
IF v_old_dept_id = p_new_dept_id THEN
    SIGNAL SQLSTATE '45000'
    SET MESSAGE_TEXT = 'Error: Employee is already in this department.';
END IF;
-- Transfer employee
UPDATE employee
SET department_id = p_new_dept_id
WHERE employee_id = p_emp_id;
COMMIT;
SELECT 'Employee transferred successfully.' AS message;

```

```

END //
DELIMITER ;

-- =====
-- 4. TRIGGER FOR SALARY VALIDATION
-- =====

DELIMITER //
CREATE TRIGGER before_employee_insert
BEFORE INSERT ON employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Salary cannot be negative.';
    END IF;
END //
DELIMITER ;

-- =====
-- 5. TRIGGER FOR SALARY UPDATE VALIDATION
-- =====

DELIMITER //
CREATE TRIGGER before_employee_salary_update
BEFORE UPDATE ON employee
FOR EACH ROW
BEGIN
    IF NEW.salary < 0 THEN
        SIGNAL SQLSTATE '45000'
        SET MESSAGE_TEXT = 'Error: Salary cannot be negative.';
    END IF;
END //
DELIMITER ;

-- =====
-- 6. AUDIT TRIGGER
-- Logs department transfers
-- =====

```

```

DELIMITER //

CREATE TRIGGER after_employee_update
AFTER UPDATE ON employee
FOR EACH ROW
BEGIN
    IF OLD.department_id <> NEW.department_id THEN
        INSERT INTO employee_audit
        (
            employee_id,
            old_department_id,
            new_department_id,
            old_salary,
            new_salary,
            action_type
        )
        VALUES
        (
            OLD.employee_id,
            OLD.department_id,
            NEW.department_id,
            OLD.salary,
            NEW.salary,
            'DEPARTMENT TRANSFER'
        );
    END IF;
END //

DELIMITER ;

-- =====
-- 7. TESTING
-- =====

-- Test 1: Valid transfer
CALL transfer_employee(101, 3);

-- Check employee
SELECT * FROM employee WHERE employee_id = 101;

```

```
-- Check audit
SELECT * FROM employee_audit;

-- Test 2: Invalid employee
CALL transfer_employee(999, 2);

-- Test 3: Invalid department
CALL transfer_employee(102, 999);

-- Test 4: Employee already belongs to department
CALL transfer_employee(103, 1);

-- Test 5: Negative salary during INSERT
INSERT INTO employee
VALUES (106, 'Test Employee', 1, -5000);

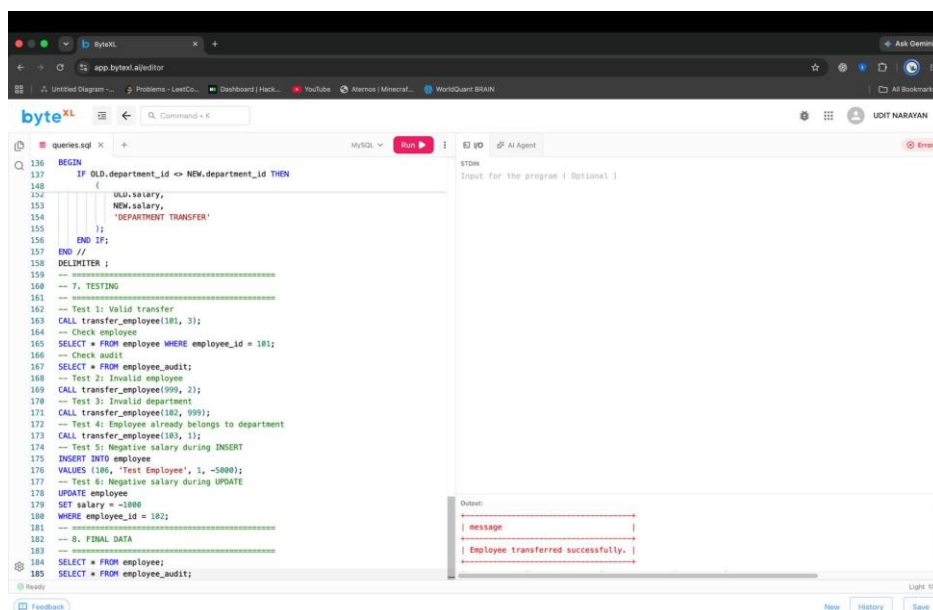
-- Test 6: Negative salary during UPDATE
UPDATE employee
SET salary = -1000
WHERE employee_id = 102;
```

```
-- =====
-- 8. FINAL DATA
-- =====
```

```
SELECT * FROM employee;
SELECT * FROM employee_audit;
```

OUTPUT:

1: SUCCESSFUL EMPLOYEE TRANSFER:



The screenshot shows a MySQL IDE interface with a SQL script in the editor and its output in the right-hand pane. The script includes a stored procedure for transferring employees, followed by several test cases. The output pane shows the results of the tests, indicating that the employee transfer was successful.

```
136 BEGIN
137 IF OLD.department_id <> NEW.department_id THEN
138 (
139     OLD.salary,
140     NEW.salary,
141     'DEPARTMENT TRANSFER'
142 );
143 END IF;
144 END //
145 DELIMITER ;
146 -- =====
147 -- 7. TESTING
148 -- =====
149 -- Test 1: Valid transfer
150 CALL transfer_employee(101, 3);
151 -- Check employee
152 SELECT * FROM employee WHERE employee_id = 101;
153 -- Check audit
154 SELECT * FROM employee_audit;
155 -- Test 2: Invalid employee
156 CALL transfer_employee(999, 2);
157 -- Test 3: Invalid department
158 CALL transfer_employee(102, 999);
159 -- Test 4: Employee already belongs to department
160 CALL transfer_employee(103, 1);
161 -- Test 5: Negative salary during INSERT
162 INSERT INTO employee
163 VALUES (106, 'Test Employee', 1, -5000);
164 -- Test 6: Negative salary during UPDATE
165 UPDATE employee
166 SET salary = -1000
167 WHERE employee_id = 102;
168 -- 8. FINAL DATA
169 -- =====
170 SELECT * FROM employee;
171 SELECT * FROM employee_audit;
```

Output:

```
1 message
1 Employee transferred successfully.
```

2: INVALID EMPLOYEE / DEPARTMENT VALIDATION:

The screenshot shows the byteXL MySQL IDE interface. The SQL editor on the left contains a PL/SQL block with a trigger and several test cases. The output window on the right displays the results of the execution, including a message 'Employee transferred successfully.' and a table showing the audit details of the transfer.

```
136 BEGIN
137   IF OLD.department_id <> NEW.department_id THEN
138     (
139       OLD.salary,
140       NEW.salary,
141       'DEPARTMENT TRANSFER'
142     )
143   END IF;
144 END //
145 DELIMITER ;
146 -- 7. TESTING
147 -- =====
148 -- Test 1: Valid transfer
149 CALL transfer_employee(181, 3);
150 -- Check employee
151 SELECT * FROM employee WHERE employee_id = 181;
152 -- Check audit
153 SELECT * FROM employee_audit;
154 -- Test 2: Invalid employee
155 CALL transfer_employee(999, 2);
156 -- Test 3: Invalid department
157 CALL transfer_employee(182, 999);
158 -- Test 4: Employee already belongs to department
159 CALL transfer_employee(181, 1);
160 -- Test 5: Negative salary during INSERT
161 INSERT INTO employee
162 VALUES (106, 'Test Employee', 1, -50000);
163 -- Test 6: Negative salary during UPDATE
164 UPDATE employee
165 SET salary = -1000
166 WHERE employee_id = 182;
167 -- 8. FINAL DATA
168 -- =====
169 SELECT * FROM employee;
170 SELECT * FROM employee_audit;
```

Output:

message
Employee transferred successfully.

employee_id	employee_name	department_id	salary
181	Rahul	3	50000.00

audit_id	employee_id	old_department_id	new_department_id	old_salary	new_salary	action_type	action_time
1	181	1	3	50000.00	50000.00	DEPARTMENT TRANSFER	2026-10-06 18:22:15

3: SALARY VALIDATION TRIGGER:

The screenshot shows the same byteXL MySQL IDE interface as before, but the output window now displays error messages for the invalid test cases. The SQL code in the editor is identical to the previous screenshot.

Output:

message
Error: Transfer failed.

message
Error: Transfer failed.

message
Error: Transfer failed.

message
Error: Transfer failed.

4: SALARY AUDIT LOG:

